GOOGLE CLOUD SKILLS | CHALLENGE LAB REPORT

Configure Service Accounts and IAM Roles for Google Cloud

Challenge Lab · Lab ID: ARC134

PREPARED BY	Yabesh J
DATE COMPLETED	20 October 2026
PLATFORM	Google Cloud (IAM, Compute Engine, BigQuery)
LEVEL / DURATION	Introductory / 15 minutes
FINAL SCORE	100 / 100

1. Overview

A **challenge lab** gives a scenario and a list of tasks, without step-by-step instructions. The tasks are completed with skills from earlier labs, and an automated scorer checks the result. This one is set in the role of a junior cloud architect who has to use service accounts, configure IAM permissions with the gcloud CLI, add a custom role, and use the client libraries to reach BigQuery from a service account. All resources were created in the **us-east1** region and **us-east1-c** zone.

2. Tasks

- Create a service account named **devops** with the gcloud CLI
- Grant it IAM roles (*iam.serviceAccountUser* and *compute.instanceAdmin*) and store its email in a variable
- Create a VM named **vm-2** with that service account attached, and check its permissions from inside the VM
- Create a custom role from a YAML file with the permissions cloudsql.instances.connect and cloudsql.instances.get
- Create a service account **bigquery-qwiklab** with BigQuery Data Viewer and BigQuery User, attach it to a VM named **bigquery-instance**, and query a BigQuery public dataset with Python

3. Concepts and Resources

Concept	Details
Service account	An identity used by applications and VMs rather than people. A VM can run as a service account and use its permissions.
roles/iam.serviceAccountUser	Lets a principal attach and act as a service account, for example when creating a VM that runs as it.
roles/compute.instanceAdmin.v1	Permissions to create and manage Compute Engine instances.
BigQuery Data Viewer / User	Data Viewer reads datasets and tables. BigQuery User can run jobs such as queries.
Access scope	cloud-platform scope on a VM lets the service account use Google Cloud APIs, with IAM controlling what it may do.
Custom role	A user-defined set of permissions, created here at project level from a YAML file.

Resource	Details
Project	qwiklabs-gcp-01-ea2f3b075960
Service account devops	devops@qwiklabs-gcp-01-ea2f3b075960.iam.gserviceaccount.com
VM vm-2	e2-medium, us-east1-c, Debian 12, runs as devops (10.142.0.2)
Custom role	cloudsqlCustomRole – Cloud SQL Custom Role, GA: cloudsql.instances.connect, cloudsql.instances.get
Service account bigquery-qwiklab	Roles: BigQuery Data Viewer, BigQuery User
VM bigquery-instance	e2-medium, us-east1-c, Debian 12, runs as bigquery-qwiklab (10.142.0.3)

All resources were temporary and were removed when the lab ended.

4. Step-by-Step Walkthrough

Setup and Task 2 – Create the devops service account

I stored the project, region and zone in variables, set the gcloud defaults, and created the **devops** service account. Listing the service accounts then shows devops next to the lab's own service accounts.

```
export PROJECT_ID=$(gcloud config get-value project)
export REGION=us-east1
export ZONE=us-east1-c
gcloud config set compute/region $REGION
gcloud config set compute/zone $ZONE

gcloud iam service-accounts create devops \
  --display-name="devops" --project=$PROJECT_ID
gcloud iam service-accounts list --project=$PROJECT_ID
```

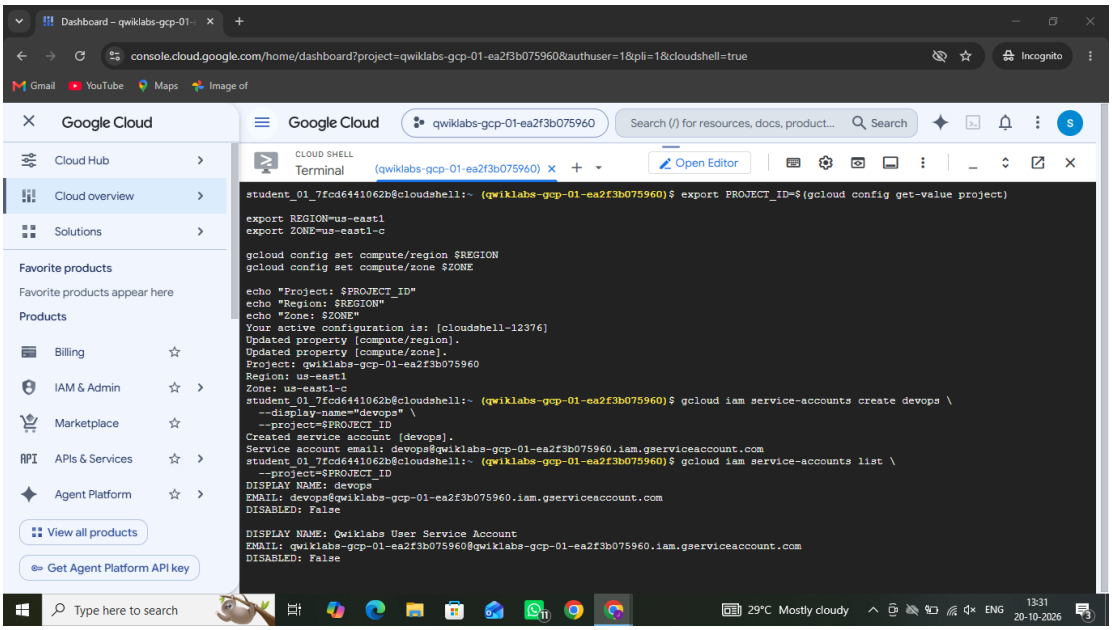


Figure 1: Variables and defaults set, service account devops created (devops@qwiklabs-gcp-01-ea2f3b075960.iam.gserviceaccount.com) and listed.

Task 3 – Grant IAM permissions

I saved the service account's email in a variable called **SA** and added the **roles/compute.instanceAdmin.v1** role to it at project level. The project already had conditional bindings, so gcloud asked which condition to use; I chose option 2, **None**.

```
export SA="devops@${PROJECT_ID}.iam.gserviceaccount.com"
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:$SA" \
  --role="roles/compute.instanceAdmin.v1"
```

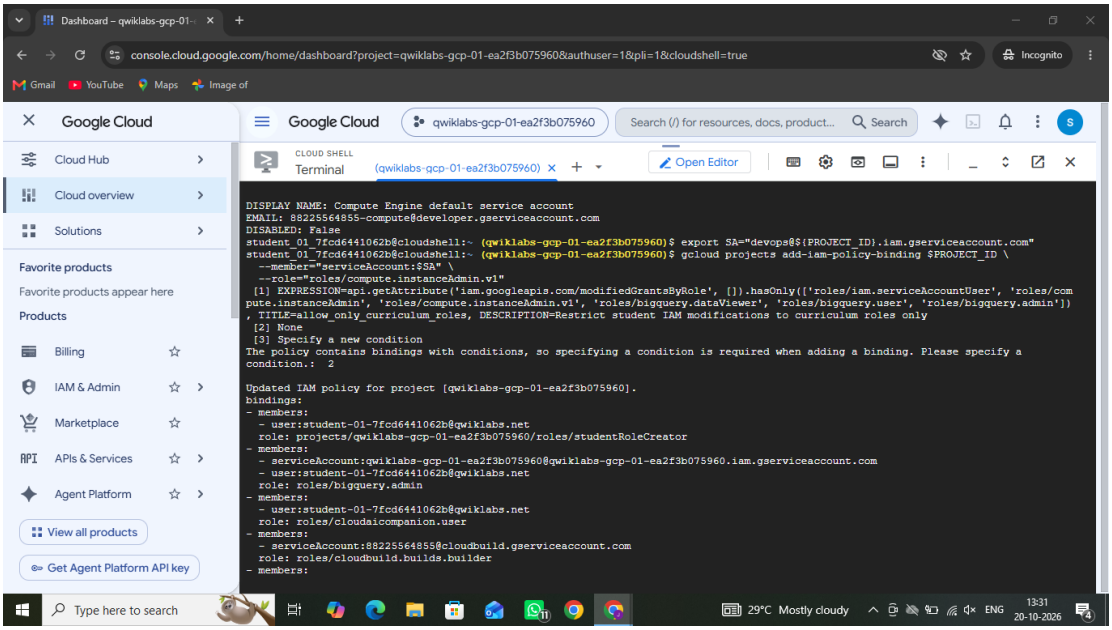


Figure 2: SA variable exported and the instanceAdmin role binding added. gcloud asks for a condition because the policy has conditional bindings, and None (option 2) is selected.

Next I checked the binding for devops and added the **roles/iam.serviceAccountUser** role, first on the service account itself and then at project level:

```
gcloud projects get-iam-policy $PROJECT_ID --flatten="bindings[].members" --filter="bindings.members:devops"

gcloud iam service-accounts add-iam-policy-binding $SA \
  --member="serviceAccount:$SA" \
  --role="roles/iam.serviceAccountUser" --project=$PROJECT_ID

gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:$SA" \
  --role="roles/iam.serviceAccountUser" --condition=None
```

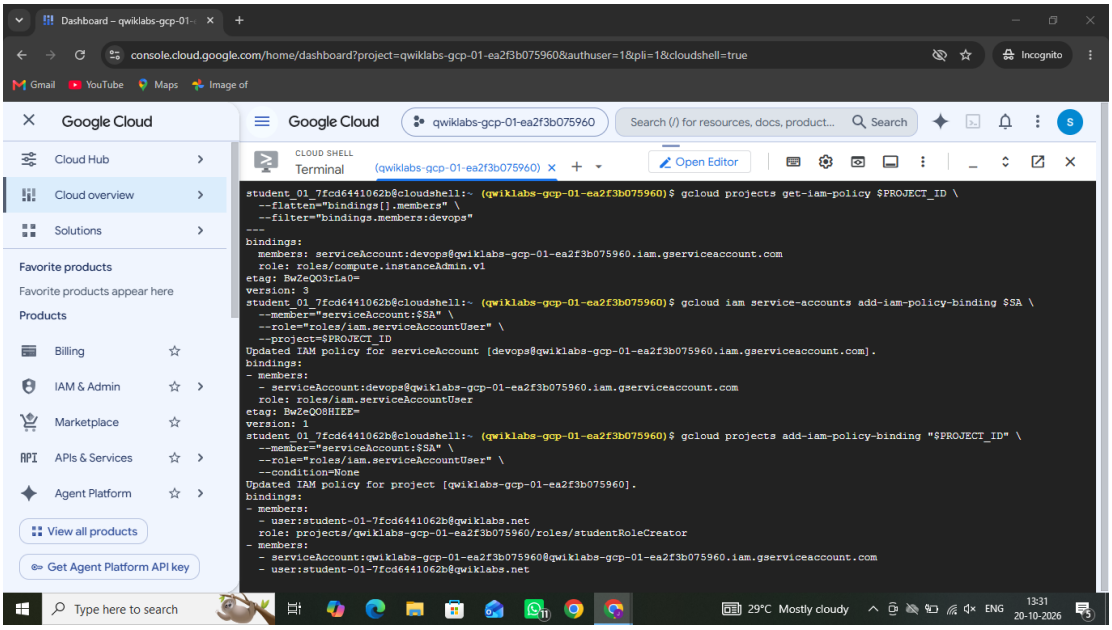


Figure 3: The policy shows devops with roles/compute.instanceAdmin.v1. The serviceAccountUser role is then added to the service account and to the project (Updated IAM policy).

Task 4 – Create a VM that uses the service account

A table of the roles bound to the service account shows both *compute.instanceAdmin.v1* and *iam.serviceAccountUser*. I then created **vm-2** to run as devops, and described it to confirm the service account and scope.

```
gcloud compute instances create vm-2 \
  --project="$PROJECT_ID" --zone=us-east1-c --machine-type=e2-medium \
  --service-account="$SA" --scopes=https://www.googleapis.com/auth/cloud-platform \
  --image-family=debian-12 --image-project=debian-cloud

gcloud compute instances describe vm-2 --zone=us-east1-c \
  --format="yaml(name,status,serviceAccounts)"
```

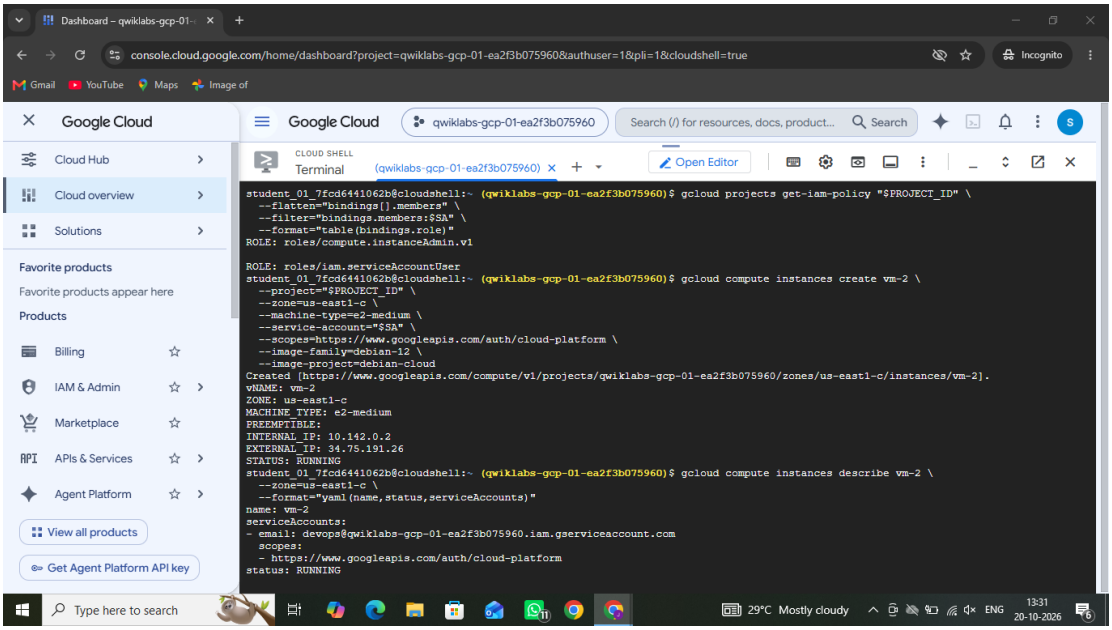


Figure 4: Roles on devops confirmed (instanceAdmin.v1 and serviceAccountUser), vm-2 created and RUNNING in us-east1-c, and describe showing devops as its service account with the cloud-platform scope.

I connected to vm-2 over SSH (gcloud generated an SSH key on first use) and tested its permissions from inside the VM:

```
gcloud compute ssh vm-2 --zone=us-east1-c
```

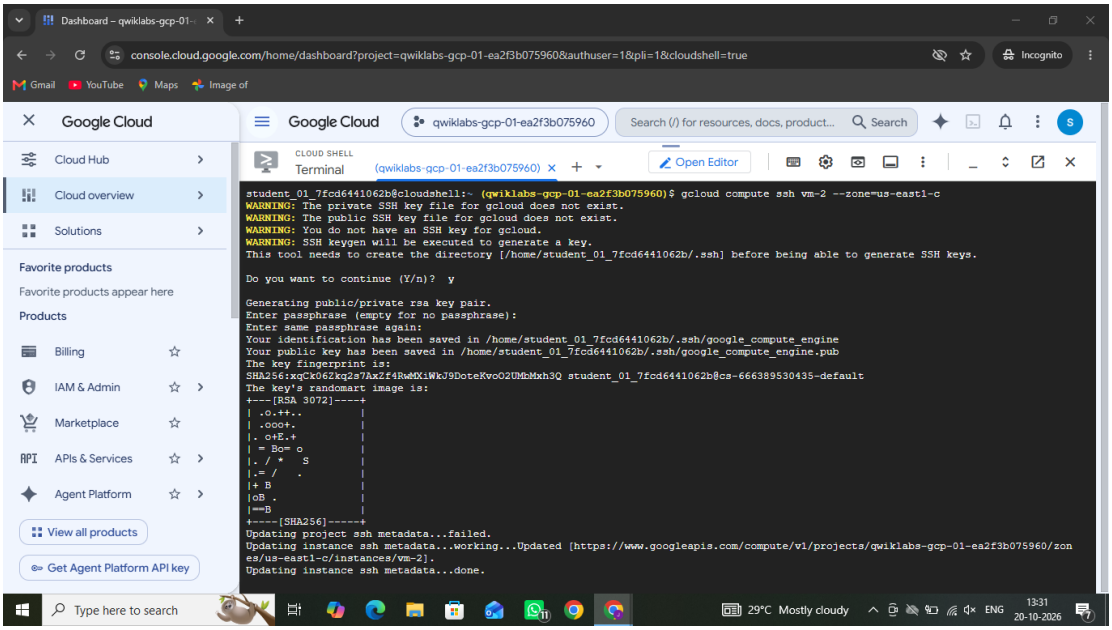


Figure 5: SSH key pair generated and metadata updated for the first connection to vm-2.

Inside vm-2, `gcloud auth list` shows that the active account is the devops service account, and `gcloud compute instances list` succeeds and shows vm-2. This confirms the VM acts through the service account's permissions.

Task 5 – Create a custom role from a YAML file

Back in Cloud Shell I wrote *role-definition.yaml* and created a project-level role from it:

```
cat > role-definition.yaml <<'EOF'
title: "Cloud SQL Custom Role"
description: "Custom role for Cloud SQL access"
stage: "GA"
includedPermissions:
  - cloudsql.instances.connect
  - cloudsql.instances.get
EOF

gcloud iam roles create cloudsqlCustomRole --project="$PROJECT_ID" --file=role-definition.yaml
```

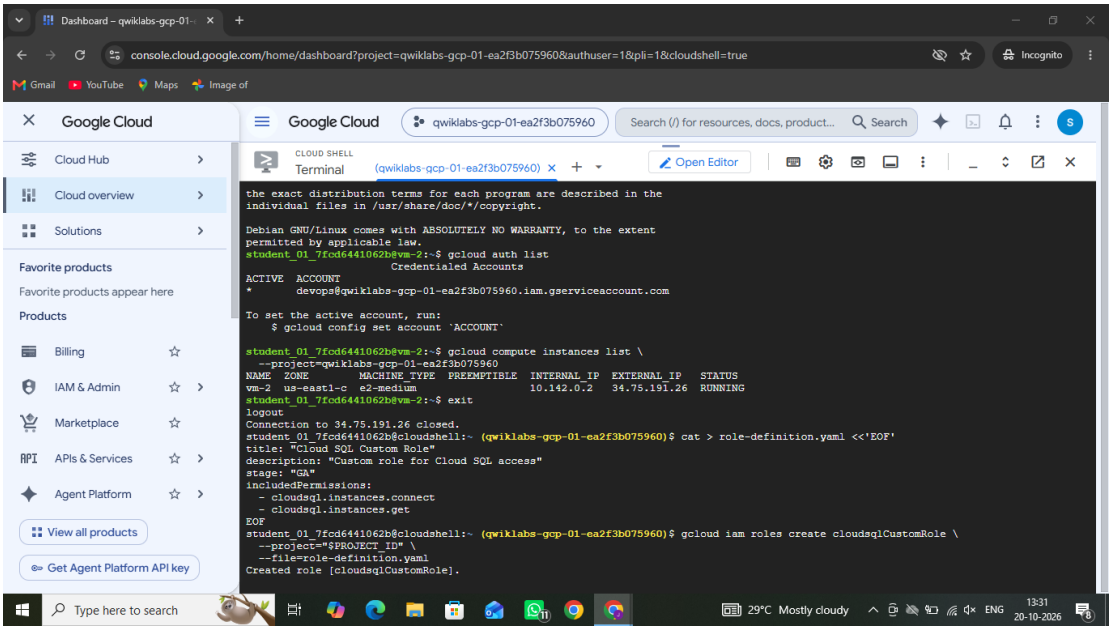


Figure 6: On vm-2, the active account is the devops service account and instances list works. Then role-definition.yaml is written and the custom role is created (Created role [cloudsqlCustomRole]).

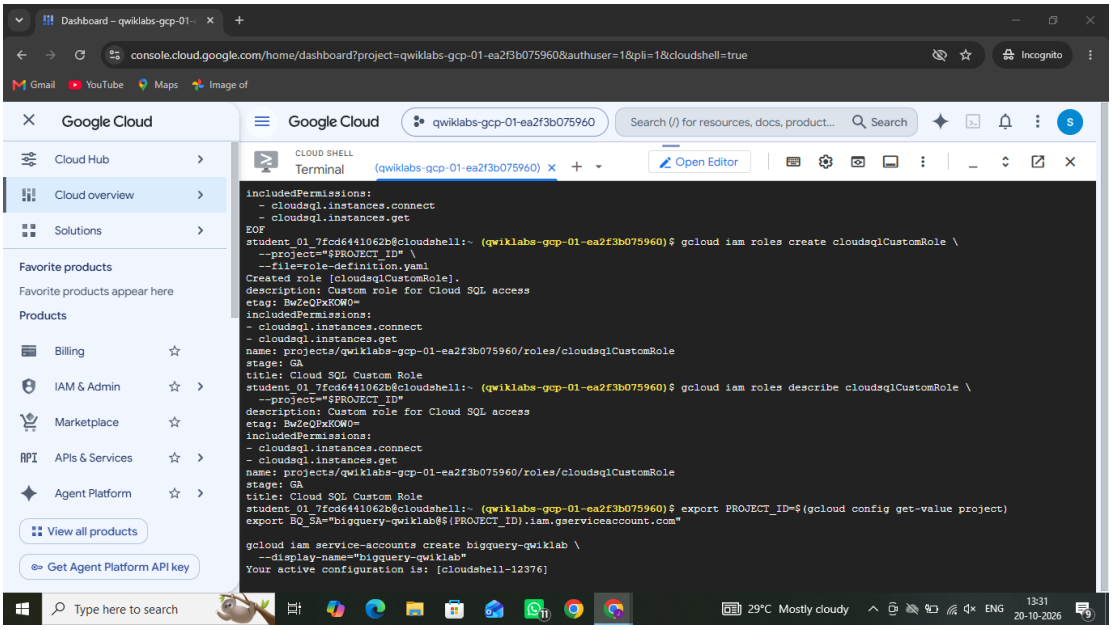


Figure 7: describe confirms cloudsqlCustomRole (stage GA, with the two cloudsql permissions). The bigquery-qwiklab service account is then created.

Task 6 – Access BigQuery from a service account

I created the **bigquery-qwiklab** service account and gave it the BigQuery Data Viewer and BigQuery User roles. The updated policy lists *roles/bigquery.dataViewer* and *roles/bigquery.user* for it.

```
export BQ_SA="bigquery-qwiklab@${PROJECT_ID}.iam.gserviceaccount.com"
gcloud iam service-accounts create bigquery-qwiklab --display-name="bigquery-qwiklab"
gcloud projects add-iam-policy-binding "$PROJECT_ID" --member="serviceAccount:$BQ_SA" \
  --role="roles/bigquery.dataViewer" --condition=None
gcloud projects add-iam-policy-binding "$PROJECT_ID" --member="serviceAccount:$BQ_SA" \
  --role="roles/bigquery.user" --condition=None
```

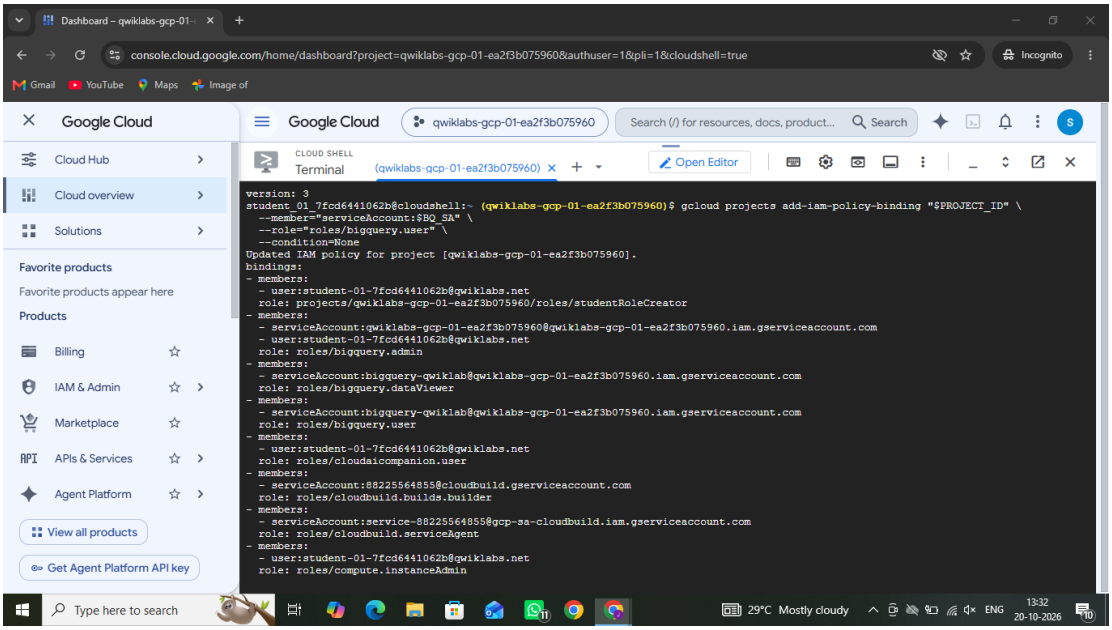


Figure 8: bigquery.user role added for bigquery-qwiklab. The updated policy shows both bigquery.dataViewer and bigquery.user bound to it.

I created **bigquery-instance** to run as that service account and connected to it. The first two SSH attempts were refused while the VM's SSH service was still starting, then the connection succeeded.

```
gcloud compute instances create bigquery-instance --zone=us-east1-c --machine-type=e2-medium \
  --service-account="$BQ_SA" --scopes=https://www.googleapis.com/auth/cloud-platform \
  --image-family=debian-12 --image-project=debian-cloud
gcloud compute ssh bigquery-instance --zone=us-east1-c
```

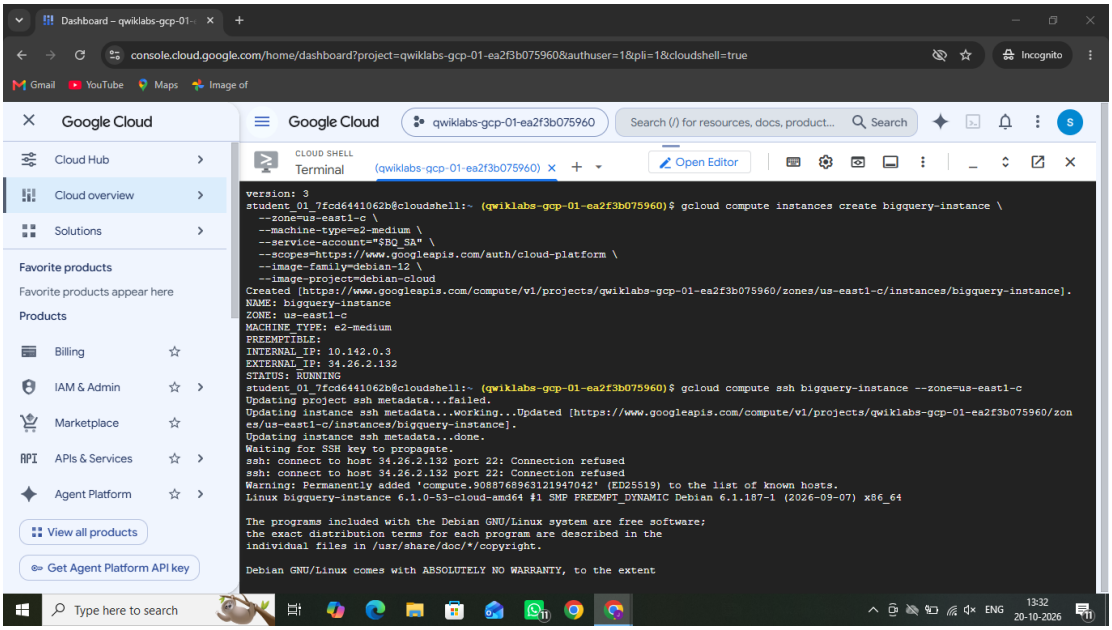


Figure 9: bigquery-instance created and RUNNING in us-east1-c with the bigquery-qwiklab service account, and the SSH connection established.

I installed the Python dependencies in a virtual environment: the BigQuery client library, pandas and db-dtypes.

```
sudo apt-get update
sudo apt-get install -y python3-pip python3-venv
python3 -m venv ~/bq-env
source ~/bq-env/bin/activate
pip install google-cloud-bigquery pandas db-dtypes
```

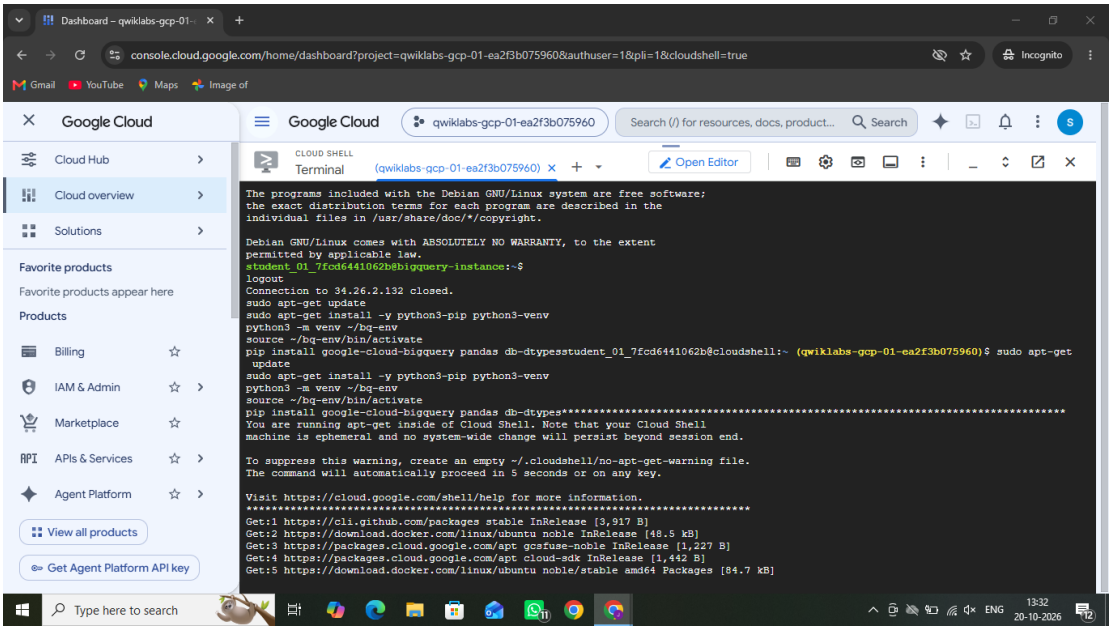


Figure 10: Dependency installation commands running (apt-get update shown in progress).

Then I wrote *query.py*, which queries the public *usa_names* dataset for the 20 most common names in Texas, and ran it with python3. One run stopped with an error: the **db-dtypes** package, which BigQuery needs to convert results to a pandas DataFrame, was not found, so the fix is to install db-dtypes in the environment that runs the script.

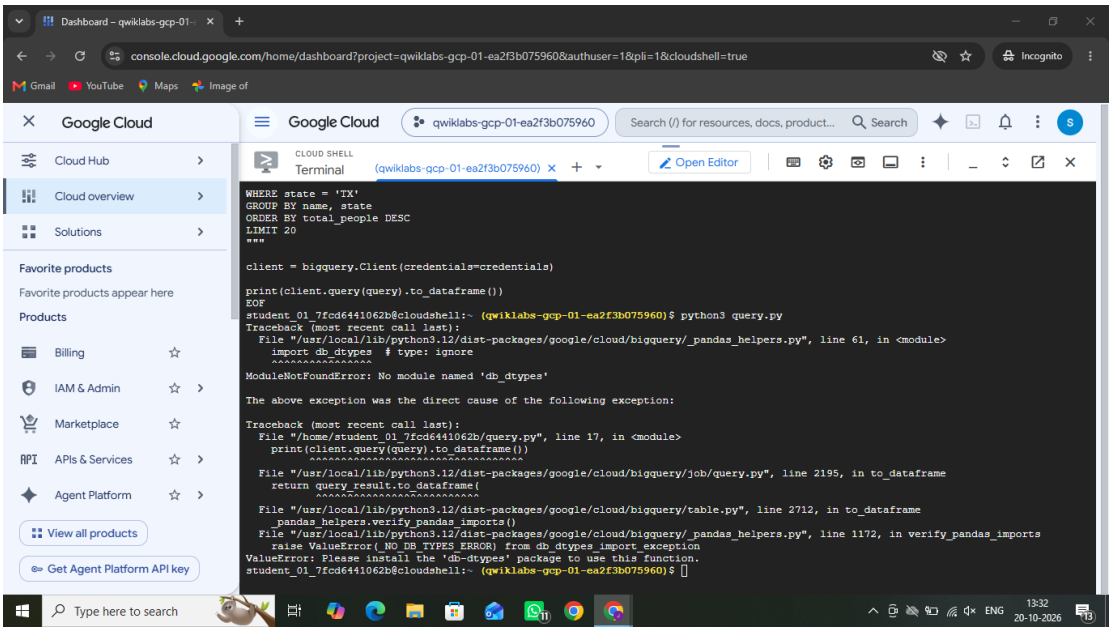


Figure 11: Running query.py returns "ModuleNotFoundError: No module named 'db_dtypes'", followed by the BigQuery message asking to install the db-dtypes package.

5. Lab Completion

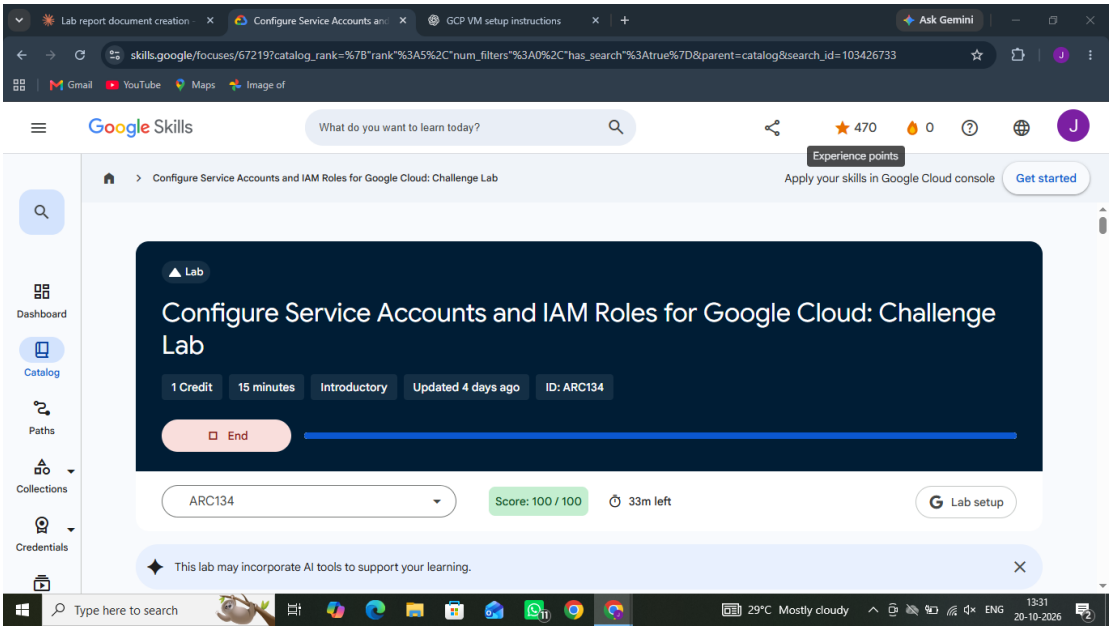


Figure 12: Lab completed with a score of 100/100.

6. Key Takeaways

- A **service account** lets a VM or application use Google Cloud APIs without a person's credentials. A VM can be created to run as one.
- Creating a VM that runs as a service account needs the **iam.serviceAccountUser** role, and the account needs roles for what the VM will do (here, **compute.instanceAdmin.v1**).
- Storing the project ID and service account email in **variables** avoids typos and keeps commands repeatable.
- Inside the VM, `gcloud auth list` shows the service account as the active identity.
- A **custom role** can be defined in YAML and created at project level with `gcloud iam roles create`.
- BigQuery access from a VM needs the right roles (Data Viewer to read, User to run queries) on the service account, and the client libraries pick up its credentials.
- Read error messages: the db-dtypes error names the missing package and how to fix it.

7. Conclusion

The lab was completed with a score of 100/100. I created and configured service accounts with gcloud, granted them IAM roles, ran VMs under them, created a custom role from a YAML file, and set up a VM and service account for querying BigQuery with the client libraries.